Citations Resolve, Humans Rule: Building Checkable Reference Documents from a Growing Corpus with Language Models, Code Checks and Recorded Human Decisions

Kostadis Roussos
Independent researcher
kostadis@gmail.com

Draft, November 2026

Spoiler warning. This paper names player characters and describes events from a running campaign, and the session summaries it cites are public. Players in the campaign read it at their own risk.

Abstract

A language model can turn a large body of records into a readable reference document in one call. The result reads well and cannot be trusted: it covers a fraction of the records, it merges and invents entities, and nothing in it can be checked without re-reading the sources. A companion survey traced twelve attempts to build such documents for a long-running tabletop campaign and ended with a pipeline that reads the game master's reviewed session summaries directly and uses a model only to write prose. This paper is the sequel. It reports what it took to make that pipeline's output something a person can rely on, and the recorded failures that shaped it. The corpus is 67 reviewed session summaries (409 scenes), and the documents are the four "grounding" documents and the character dossiers that the next session is prepared from.

The system rests on five rules. A model's output never reaches another model call without a code check between them. Identity (who is who, which thread is which) is declared in a registry and matched exactly, never by similarity. Code builds every table, timeline and ordering; a model writes only prose, from notes that code has checked and routed. A finished draft is annotated, never rewritten. A draft becomes a document only when a named person signs the exact bytes. Each rule is enforced by a test that fails the build, not by an instruction in a prompt.

On the full corpus, the survey's endpoint, a single-call draft by a frontier model that the GM had judged better than every earlier attempt, cited 22% of scenes; the chunked, code-checked pipeline on a local open-weight model cited 99%, named three times as many registered entities, and corrected a factual error the single call could not see. The code check dropped 118 of 3,219 model notes for a missing citation, a citation outside the model's input, or a quotation that was not verbatim. We also report what the checks do not catch. A citation that resolves is not a citation that supports the line: one published dossier carried an invented fact with a valid citation. A model asked to repair flagged errors helped in about a third of cases, did nothing in a third and did harm in a third, so the system detects and annotates and leaves repair to a person. We describe an earlier design, a per-session workflow engine with hash-bound approval gates, that had the right gate semantics and failed on the cost of review, and what the present system kept from it. The method is not specific to games; we discuss where it applies and what it needs.

1 Introduction

Many organisations keep a growing pile of dated records and need a small set of current documents that say where things stand: a patient summary over years of encounter notes, a matter summary over case filings, a runbook over incident reports, a state-of-the- project page over meeting minutes. A language model writes such a document quickly. The difficulty is that the reader has no way to tell which sentences to believe.

This paper reports on one such system, built by one person for his own use, over a corpus with unusually good ground truth. The author runs long tabletop role-playing campaigns as game master (GM). Each session is recorded, transcribed and written up as a structured summary that the GM reviews. Before the next session the GM needs four documents: what has happened in the world, what content is complete, where each character stands, and what each faction and non-player character (NPC) wants. An error in these documents surfaces at the table, in front of the players, as a character who acts against the record. The GM's stated standard is: "I cannot afford a moment where my own lack of precision breaks the fourth wall."

The design principle was written down early: "Never feed an LLM's output to another LLM without human review if you want precision." The first systems honoured it by convention. They failed in ways that were quiet and repeated, and each failure left an issue in the tracker with numbers attached. This paper is built from that record.

Relation to the companion papers. A survey by the same author [Roussos, 2026b] catalogues twelve approaches to the grounding documents, from March to early October 2026, by what a pipeline reads and who makes its precision decisions (ordering, attribution, identity, scope). It ends with *summary-native* grounding: validate the reviewed summaries, parse their scenes and entity sections with no model call, and have a model write the four documents in one call each. The GM promoted those drafts. A second paper [Roussos, 2026c] tests non-generative decision models in the same pipeline's judgment steps and finds them useful as filters and flags, never as deciders, and that "a ruling should be made once" and read back on every run.

The survey's endpoint is this paper's starting point. Within two days of the promotion, measurement showed that the single-call drafts read well and covered a fifth of the record (Section 8). The work reported here is what followed: chunked extraction with a code check between every model call, dossiers with mechanical verification, a ledger for the GM's rulings, a review store, a promotion gate bound to signed digests, and an incremental update path. We also report a design the survey does not cover, because it was built for the narration side of the pipeline and never merged: a per-session workflow engine whose gate semantics reappear in the present system. Where a failure class below is already documented in the survey, we cite it and give only the incidents that came after.

Claims. We are careful about what we claim. The documents this system produces are not proven true. They are *checkable*: every content line carries a citation that code has resolved to a passage the model was shown, every quotation has been compared to that passage, every entity has been matched to a registry entry, and every published file is bound by digest to a named person's sign-off. The class of error that remains, a resolving citation under a claim the passage does not support, is named, measured where we could measure it, and routed to the person. The reader of the document is told to treat it as an index into the records, and we show that a reader who does so finds more errors at no extra cost.

Contributions.

- A failure taxonomy of ten classes drawn from the project's issue tracker and experiment logs, each with its recorded incidents and the structural countermeasure adopted (Section 3).
- Five design rules and the mechanism that enforces each, including build-failing tests that walk the syntax tree to prove a module cannot call a model (Section 4).
- An account of a first design that stalled: a per-session workflow engine with hash-bound gates, which a coding agent invented when told to read the code and the open issues and build something that would work. We analyse why correct gate semantics were not enough (Section 5).
- The present system: a map, check and reduce pipeline over reviewed summaries, with a rulings ledger, a review store, a promotion gate and an incremental update path (Section 6), driven by agent skills that keep the same rules (Section 7).
- Measurements on a 67-chapter corpus across five models, including two negative results: model repair is unsafe, and quote fidelity in the reduce step varies from 61% to 94% by model (Section 8).

2 Setting

The corpus. A campaign is a directory of session summaries, one per session, in a fixed schema: a synopsis, numbered scenes with bullet events, and sections for locations, NPCs, items and memorable moments. Each summary is produced from a session recording by an external service and a model pass, and is reviewed by the GM before anything is built from it. The summaries are the authority. Where a generated document and a summary disagree, the summary wins and the document is wrong.

The main evaluation corpus is one campaign, chapters 2 to 70: 67 files and 409 scenes. A deterministic parse of it finds 590 NPC entries, 319 locations, 394 items and 301 spells (`specs/031-summary-native-grounding/session-log.md`). The entity registry for this campaign holds 1,012 entities. Other campaigns in the same repository have between 11 and about 86 sessions; three were in active use during the work.

The documents. Four grounding documents are rebuilt from the summaries: `world_state` (factions, places, threats, key NPCs), `campaign_state` (completed content, NPC status, open and closed threads, an audit of tracked claims), `party` (each player character and the group) and `planning` (NPC dossiers, active plots, a threat tracker). Per-NPC dossiers are built separately and published one by one. The documents are read by the GM and by a model during session preparation.

The authorities. Three small hand-maintained files hold identity. The *entity registry* holds each entity's canonical name, its approved alternate names, and explicit "these two are distinct" rulings. `players.yaml` holds who plays which character. The *thread registry* holds the named plot threads, written only when the GM ratifies one. A fourth file, the *authority ledger*, holds the GM's rulings where the record is wrong or silent.

The actors. A command-line pipeline in Python does the work. A coding agent runs it through *skills*: procedure files that tell the agent which commands to run, which checks to read and where to stop for the person. The models are a frontier model through an API or subscription and open-weight models served on two local machines. The person is one GM, who reviews mostly from a phone.

3 The failure record

Between August and November 2026 the repository received 600 commits, of which 94 carry a `fix` prefix (our count from `git log`). The issues behind them fall into ten classes. Table 1 lists them. We give each a short account here, because the design rules in the next section are these failures read backwards.

Scope decisions (class 1). The earliest recorded incident is a cleanup pass that was asked to remove game mechanics from narration and decided for itself what counted. It rewrote spell names: *Speak with Dead* became "question the dead" and *Magic Missile* became "a lance of unerring force light" in one scene and "darts of pure force" in another (#151). It was found only because someone happened to read the output. In a later experiment a model was asked to fill eleven gaps it had itself marked as "the GM said this". It filled all eleven, grew the scene from 2,223 to 2,804 words, and gave both of the GM's lines to a player character, on the one passage the markers had been protecting.

Identity by similarity (class 4). This is the best-documented class; the survey reaches the same conclusion for grounding ("identity is the precision decision") from the atomic-fact and knowledge-graph approaches. On the narration side the incidents are these. A voice file was matched to a character by a filename prefix. When a character was renamed, the rename was applied in two stores and left stale in four; 1,608 files used one spelling and 75 the other (campaigns#175). A speaker-label parser skipped any label in brackets because brackets usually marked stage directions; in one session this discarded 39 turns of one character and 21 of another (#453). A code-built status table keyed rows on the literal name and listed "Ilvara Mizzrym: Alive" and "Ilvara: Dead" as two people. In an audit of one campaign's registry, 22 of 434 recorded aliases were transcription errors that had been promoted to approved names.

Silent delivery (class 6). The survey names silent success the dominant failure mode and asks for positive controls. One form of it is worth a separate class. A rule that never reaches the model fails without a symptom when the rule agrees with what the model would do anyway. The project's test for delivery is therefore a rule the model would not follow unprompted: after a genre rulebook that requires present tense was correctly delivered, one scene's verb count went from 31 past and 4 present to 7 past and 30 present. Before the fix, 3 of 62 rendered scenes were in the required tense (`docs/design/GenreRulebook_implementation.md`).

Cached breakage (class 3). One extraction run lost 250 of its 309 dropped notes to a single cause, a missing citation, and one chapter kept 0 notes of 42 (#532). The process exited 0. A separate verifier reported no problems on a chapter that held 40 quotations in curly quotes, because it looked only for block quotes and found none to check. The project's rule is now that zero items checked is zero coverage, not a pass.

4 Design rules

R1. No model output reaches a model call unchecked. Between any two model steps there is a deterministic check and a file on disk. The map step writes notes; code checks them; the reduce step reads only the survivors. Parallel batches of the same step never see each other's output. No document's draft is given as context to another document's draft. A cached model output is

re-checked from its raw text on every run, so a stricter check applies to old outputs without a new call.

R2. Identity is declared and matched exactly. A line is attributed to an entity only if it uses a name or approved alternate name in the registry, compared by equality after case folding where the registry allows it. A name form that could mean two entities is withheld from linking and cannot be ruled to one of them. A name form that is an ordinary English word (an NPC called "Stool", 122 occurrences) is withheld until the GM rules. On the main corpus, linking 222 NPCs to 1,619 scenes withheld 8 ambiguous and 11 generic forms. An alias is an approved alternate name, never a misspelling; a misspelling is fixed in the source file.

R3. Code owns structure; the model owns prose. Ordering, selection, counting and status are computed. The timeline is model notes stitched in chapter order by code. An NPC's current status is the latest cited status row for that registry entity. Which threads are open and in what order is read from the thread registry. The model receives the notes that code routed to one section and writes that section. Quantities the GM owns are never stated: the documents list candidate events for a character's arc score with a verbatim trigger, and a line that states a value is dropped by rule.

R4. Annotate, never rewrite. After a draft is written, code looks for newer information about each line's subject, later status for each named NPC, and quotations that are not verbatim in the cited chapter. What it finds is appended as a marked sub-line under the original. The original line's text is never changed. The only removals are structural: a player character listed as an NPC.

R5. A draft is not a document. Every generated file is written under a range-specific draft directory. It becomes a document only by publication, and publication requires a named person's sign-off bound to the digest of the exact bytes. A later change to the draft makes the sign-off stale. A published file that no longer matches its recorded digest has been edited by hand, and the publisher refuses to overwrite it.

Two supporting rules. *Fix at the source*: an error in a draft is fixed in the summary, the registry, an authored file or a ruling, because a draft is regenerated and a patch to it is lost. *Refuse, do not fall back*: when a required input is missing, the tool stops with the command that supplies it. The project's record has several cases where a "graceful" fallback was the defect.

Enforcement. A rule stated only in a prompt or a contributor guide is followed until it is not. Each rule above has a test that fails the build (Table 2). The most used technique is a walk over the syntax tree of the modules that must not call a model, asserting that they import no model client and call no model function. Twenty-eight modules of the pipeline are guarded this way; the five modules that do call a model are named exceptions. Other guards take a digest of the whole document tree before and after a run, plant a canary string in secret fields and search every prompt and output for it, or compare an assembled prompt byte for byte to a frozen copy.

5 A first design: the session cycle

The first attempt to make the rules structural was a workflow engine for one session's production, from transcript to published narration to campaign memory. Its origin is worth recording. The

author was stuck on the project's overall implementation strategy. On the day OpenAI released its Astra model, with free tokens on offer, he gave it a single instruction through the Codex agent, out of curiosity about what it could do: review the code base and the open issues and come up with a solution. The exact prompt was not kept; it was deliberately vague. He then walked away. The agent's plan records a 59-issue allocation as its governing scope. What it produced, with no direction as to form, was the system described here: five stacked branches, 43 files, about 4,000 lines and 38 tests, all committed within one day in September 2026. No person designed the gate model below. An agent derived it, in one shot, from a tracker full of the failures in Section 3.

The work was never merged. Its pull requests were closed in November with the note that the approach had been superseded. We describe it because its gate semantics were sound and most of them reappear in the present system, and because the reason it stalled is the most transferable lesson in this paper.

5.1 Design

One file per session, `session_workflow.yaml`, was the authoritative record. A content-addressed store beside it kept the exact bytes of every input and output. The record held *runs* of thirteen fixed stages (prepare, capture, identify, events, remove-recap, extract, voice-smooth, no-mech, plan, narrate, release, memory, prepare-next), each with its parent stages, mandatory checks and a named human decision.

A check produced *findings*. Each finding had to state what would happen if the person approved it, rejected it or asked to discuss it. A *decision* was bound to the digest of the finding it answered, so a decision on an older version of a finding was refused. A proposed change had to match exactly one place in its target.

A *gate* was an approval bound to a fingerprint of the run's inputs, outputs, checks, decisions and dependencies. The engine refused to approve a run with a missing or failed check, an undecided finding, a finding marked "discuss", or an approved change not yet applied. A clean audit did not approve anything: "A clean audit still requires a separate, named, explicit human approval." A stage could not start until every parent was approved, and a change to any input byte made every downstream approval stale. Approved and stale were computed from the record, never stored.

Writes were journaled. Before touching any target the store wrote the before and after digests; recovery completed a write only if each target matched one of the two and otherwise refused, leaving the originals. Promotion into the campaign's documents recorded each destination's previous digest as part of what was approved, and refused if the live file had changed since.

The web interface was a thin adapter. Every operation ran in the command-line tool as a subprocess, so the editor, the command line and an agent read and wrote the same record.

5.2 Why it stalled

No stage beyond the first few ever ran on real content. The acceptance record reads: "one pending human handoff, zero model calls, zero rendered scenes". The author's account of why is about use, not correctness, and has four parts.

1. *Two surfaces.* The work was done in an agent session; the review was done in a web page. Each stage meant leaving one for the other. The walkthrough document concedes the seam: "approving a draft does not automatically start the next stage. The editor cannot run an arbitrary skill on its own."
2. *The wrong device.* The web interface was served from one desktop machine. Much of the review was being done from a phone, away from that machine.

3. *Approval heavier than the job.* Every finding needed a decision and every stage needed a named sign-off and a click. For most stages the reviewer's real task was to catch an egregious error, not to rule on each detail. Five of the last commits on the branch reduce review friction, ending with "make single-user review a one-click action".
4. *A slow interface, built by two models that disagreed.* The existing web interface had been written with one model. The agent that built the cycle pages was a different model, and it built on top of that interface without sharing its conventions for how an interface should be put together. The result was slow to use. Getting to an interface both could extend would have been an engineering project of its own, competing with the work the pages were meant to support.

The first three are about where review happens and what it costs. The fourth is about the price of a graphical surface when the builders are agents: a command-line contract is easy for two different models to agree on, and a user interface is not.

The engine treated every stage as if an error there cost the same. It does not. A garbled quotation in one scene's draft costs a re-render. A wrong status in a document that every later session reads costs trust in the whole set. A gate that charges the second price at every stage will be bypassed or abandoned, and this one was abandoned.

5.3 What carried forward

The present system kept the semantics and changed where and how often they are paid for. The correspondence below is the author's account of the design history; the repository does not document it, and the two efforts also share older project principles.

- Three verdicts, approve, reject and discuss, with "unmarked" and "discuss" both blocking, became the review store's vocabulary (Section 6.6).
- A decision bound to the digest of what was decided became the exact-draft sign-off.
- "A clean audit does not approve" became the promotion rule that item approvals, file presence and an absence of findings do not count as sign-off.
- Journal, then write, then recover by digest became the transaction model of the rulings ledger, the review store and publication.
- Promotion that records the destination's previous digest and refuses a changed live file became the publisher's hand-edit refusal.
- Staleness computed from digests became the freshness checks that refuse a stale corpus, stale notes or a stale sign-off.

Four things changed. First, there is no stage engine. The orchestrator is a skill run by the agent in the session where the work already happens, so there is one surface. Second, review happens on a phone: a single static page with no network dependency for narration gaps, and a capability link for document review. Third, the weight of approval follows the cost of error. Intermediate stages have cheap, per-item verdicts in which leaving an item unmarked is a legitimate state; the named, digest-bound sign-off is asked for once, at publication, for the four documents and for each dossier. Fourth, most of what the cycle engine asked a person to approve is now decided by code and reported, so the person reads a drop report where they previously signed a gate.

6 The present system

The pipeline is called *summary-native* because it reads the reviewed summaries directly. It is 114 Python files and about 41,000 lines (our count). The flow is:

$$\textit{validate} \to \textit{build} \to \underbrace{\textit{extract}}_{\text{model}} \to \textit{check} \to \underbrace{\textit{synth}}_{\text{model}} \to \textit{annotate} \to \underbrace{\textit{audit}}_{\text{model}} \to \textit{review} \to \textit{sign} \to \textit{publish}.$$

6.1 Corpus: validate and build

`validate` reads the summaries and writes a report. It is "a collector, not a raiser": it lists every problem and guesses at none. Ten finding codes block a build (for example a duplicate chapter number, a title that disagrees with its file, a scene identifier in the wrong chapter) and four do not (an unknown section, a possible duplicate, a stale ruling, a gap in the range). On the main corpus it reported 8 blocking and 38 non-blocking findings in under a second.

`build` parses the range into a manifest, a chronology and per-entity evidence files. It calls no model, takes 1.5 seconds on 67 files, and is byte-identical across runs. The manifest binds every summary's digest; every later step refuses to run against a corpus that has changed since.

Possible duplicates among entity headings are listed for the GM. The fix is to correct the summary. A side file can record only that two names are *not* duplicates; it cannot merge.

6.2 Map, check, reduce

Map. `extract` calls a model once per chunk, where a chunk is about 60,000 characters, usually one chapter. The model writes notes under six headings (events, concluded content, threads, NPC status, world, party). Every note must carry a citation of the form `[ch NNN / target]` and may carry quotations. One map pass feeds all four documents.

Check. Code then reads every note. A note survives only if its citation resolves to a scene or section inside the chunk that was sent, and every quotation in it is found verbatim in the cited text (exact, or equal after folding typographic quote marks). Notes in a typed section must also parse: a status row must name a status, a level row's number must appear in the cited text, a party note must name its subject. Every dropped note is written to a report with its reason. A chunk that lacks a required section is a failed chunk; it is retried once and then the run exits non-zero. A chunk whose drop count is more than three times the run's median, and at least 20, is flagged as a runaway.

Reduce. `synth` builds each document. Code writes the timeline, the completed-content list, the NPC status table, the reading contract at the top, and reference files that hold every routed note verbatim, grouped by subject. For each prose section, code selects the notes that belong to it and a model writes the section from those notes only, within a word budget that is reported and never enforced by trimming. The Key NPCs section is written from published, verified dossiers; if an NPC in scope has none, the build refuses unless the operator passes a per-run flag, and the resulting lines are marked as lacking a dossier.

Annotate. Runs automatically after reduce (rule R4). Its detectors are: newer information on the same subject, a later status for a mentioned NPC, a citation that does not resolve, a quotation that is not verbatim, and a player character in an NPC section.

Audit. The GM keeps a tracking file of claims to watch (443 items on the main corpus). Each is judged alone: code picks up to three candidate chapters, the model judges the one item against only those, and code accepts "supported" only with a citation inside the candidates and a verbatim span of at least eight characters. Everything else is recorded as not found, with the reason. The audit was first run inside the map step; it was moved out after one map call answered all 443 items in order with well-formed, in-chunk citations and no content (Section 8.5).

6.3 Dossiers

An NPC dossier is built in five commands, of which one calls a model. `npc-link` attaches scenes and moments to an NPC by exact name form. `npc-draft` writes the dossier, chunked: code chunks the evidence, a model writes notes per chunk, code checks them, code stitches them, a model writes the short prose sections, code assembles the file. `npc-verify` re-checks the finished draft and reports seven failure codes (among them a citation outside the NPC's evidence, a quotation not found, an uncited claim, and a GM-authored edit that the draft dropped). `npc-compose` adds the GM's hand-authored material. `npc-publish` is the only writer of the published file.

Two properties matter beyond the mechanics. The GM's authored file has a numbered list of manual edits, which reach the model, and a secrets section, which does not: the drafting code reads the file through a function that cannot return the secrets, and a canary test confirms it. And the publisher checks four things in order: the draft's digest equals the composed file's header, verification passed, an exact-draft sign-off exists, and the target is either new or still matches the digest recorded when it was last published.

6.4 Threads

Plot threads are the hardest identity problem in the corpus. The map step names a thread for each thread note, and the names are useless as identity: on the main corpus the checked notes carry 554 distinct thread names, of which 550 appear once. Code therefore attaches a note to a thread only by exact title or alias in the thread registry. A model step, `thread-propose`, groups the unattached notes into candidate threads; code checks each proposed group (a member claimed by two groups is removed from both) and the GM ratifies, splits, rejects or defers. There is no accept-all. On a first run with an empty registry the proposer produced 31 groups and 216 single-note proposals from 311 notes, well short of the target for grouping; the prompt tells the model to leave a note out when unsure, and the target was dropped in favour of precision.

6.5 Rulings

The record is sometimes wrong, and the GM sometimes decides something the record does not hold. Before the authority ledger, a regeneration would restore a claim the GM had already corrected. A ruling is now staged, turned into a proposal with before and after snapshots, reviewed as an exact diff, and applied to the summary itself in a journaled transaction keyed by the proposal's digest. The ruling has no effect at read time; until it is applied to the source it is an instruction and an audit record. Because the correction lives in the source, every later build inherits it.

6.6 Review

Verifier findings become review items with the exact claim text, the evidence, a category and a digest. The GM marks each approve, reject or discuss, from a phone or the command line. Decisions are appended as events with a revision number, and a write against a stale revision is refused.

Applying a decision is a separate step. Two rules guard against reading too much into an approval: mechanical success never records that the GM supports a claim, and rejecting a proposed merge of two entities does not record that they are distinct. An approval goes stale only when a fact-bearing input to the item changes, so unrelated regeneration does not erase earlier review.

6.7 Promotion

The four documents, the timeline and the reference files are published as one bundle. A dry run computes a preview digest from a chain of digests over the source selection, the claim mappings, the analysis, the resolutions and the sign-off context. Publication requires four document sign-offs, each bound to that document's bytes, and recomputes the preview under an exclusive lock; if the digest has moved, it refuses. The new generation is staged in full and activated by one atomic rename of a symbolic link, so a reader sees the old bundle or the new one and never a mixture.

The gate was built after five historical cases in which hand-copied drafts had carried errors into the live documents. A study of those cases concluded that all five "require semantic candidate extraction plus GM disposition": none could have been caught by a mechanical check alone. The gate therefore includes a claims step in which spans are mapped to structured claims and compared for contradiction, staleness, suspicion stated as fact, audience leaks and resolved items listed as active, with the rule that confirming a mapping means "this span says this," not "the campaign fact is true."

6.8 Incremental update

Rebuilding everything for one new session re-reads 70 chapters to change a few lines. The update path binds a published baseline by digest, selects the new chapter explicitly, and has a model propose edits against numbered lines. The host supplies the exact before-text, validates every target by line digest, rejects overlapping edits, and requires each edit's support to be an exact span of a selected source. The GM rules on each proposal; reconstruction applies approved edits and leaves every other byte unchanged. For a dossier, the prior history and quotations are copied byte for byte and only new blocks are appended. Publication goes through the same promotion gate. In the hand-run update that motivated the feature, ten dossiers were updated, three were new, and 96 unselected dossiers were confirmed unchanged.

6.9 Scheduling

The map step runs on two local machines. The scheduler keeps one ordered queue across endpoints, takes concurrency from the served model's declared capacity, quarantines an endpoint on a transport failure and requeues its item elsewhere, and does not quarantine on a model's rejection. On resume, the evidence of completion is a validated artifact on disk, never the journal. The scheduler's stated scope is transport only: it selects no content and accepts no finding.

7 Skills: the orchestration layer

The pipeline's commands are run by a coding agent following skills. There are 44 skills in the collection, 17 of which ship their own deterministic scripts (our count). They are where the cycle engine's role went, and they hold the same rules in a form an agent can follow.

Propose, review, apply. The collection's governing statement is: "the LLM proposes, the human rules, a deterministic step applies... collapsing it is what caused issue #151." Detection is a script, the ruling is the person's, and application is a script that checks its own preconditions.

The model's self-report is never the gate. A skill that has subagents summarise chapters runs a verifier over every output and tells the orchestrator that the subagents' own counts "are for triage, not truth." In the reference run for that skill, unconstrained summaries carried 14 untraceable quotations against 2 for the constrained ones, and 434 novel proper nouns against 18. Verifier limits are stated in the skill: one notes that it "does not catch invented claims about real entities."

Invariants in code, not in prose. The mechanics-scrubbing skill that replaced the spell-stripping pass has no pattern category for spell names: the scanner cannot propose one. The script that applies cuts to a derived layer refuses to write to any path outside that layer.

Count-checked, all-or-nothing apply. An approved edit carries the exact number of times its old text occurs. The apply step refuses on any mismatch and writes nothing in any chapter until every edit in every chapter resolves. This is how a stale finding, a second apply or an ambiguous anchor is caught.

Reading decisions back. A review page never pre-fills a verdict, because a pre-filled verdict would be exported as the person's ruling on a single save. The decision reader exits with an error when a page was never saved, so an unsaved page is never read as "approved nothing." The skills record that "done" from the person does not guarantee a save: in one batch five of eight pages were unsaved. An unanswered question is not a decision, and a timeout is not a checkpoint.

Rulings travel in a file. Subagents cannot see the conversation, so rulings the person gives mid-run are written to a file each subagent reads. A prior ruling turns a finding into a conflict to show, never a question to ask again.

The agent's cost is a design input. One skill records that subagents which inherit the orchestrator's context cost a median 1.74 million input tokens each against 0.67 million for fresh ones, and requires fresh ones. Another assigns reading and writing to a mid-size model and adjudication to the largest.

The skills change after runs. Of 137 commits to the collection, 15 are fixes and 10 record "lessons from" a named run (our count). We regard this as the system working as intended: a failure found in use becomes a rule in the procedure and, where it can, a check in a script.

8 Evaluation

All measurements were recorded by the project during development, in an experiment log (`experiments/20261007-chunked-state-docs/RESULTS.md`) and in each feature's recorded acceptance run. The corpus is a read-only copy of one campaign, chapters 2 to 70. Most arms were run once. We report them as a case study.

8.1 Coverage and correctness against a single call

The baseline is the survey's endpoint: the first summary-native pipeline's single-call draft, one call per document to Opus 5.5 with a prompt of 610 to 660 thousand characters built from scene synopses, selected dossiers and moments. These are the same drafts the GM promoted. Summary-native was the first approach in the survey whose output the GM accepted, and the GM had judged these drafts "much better than the other ones we have built" [Roussos, 2026b]. That judgment was right about the prose and silent on coverage, which is the point of the comparison: a well-written draft is not evidence of how much of the record it read. The chunked arm is 60 map calls and 9 reduce calls to a local open-weight model (`qwen3.8-flash-next`). Table 3 gives the comparison.

The single call stated the party's level as 8 with "no later level-up". The record says level nine, in a scene bullet. The single call's prompt held synopses, not bullets, so it could not have known. The chunked pipeline reports level 9 with its citation.

On 16 facts verified by hand, the single call got 12, the first chunked model 10 and a second chunked model 12, with different misses: the single call missed facts found only in bullets, and the chunked runs over-supported loosely matched audit items. The chunked documents were also far too long to use: `world_state` was ten times the baseline. A budgeted reduce with the notes kept verbatim in reference files brought it to 16.5K characters, and the acceptance run of the shipped feature produced a 25,396-character `world_state` with 15 of 15 quotations verbatim and a `campaign_state` with 34 of 34.

8.2 What the code check drops

Table 4 gives the drops in two recorded acceptance runs of 60 chunks each. Most drops are a missing citation or a row that does not parse. Few are a quotation that is not in the source. We read this as the check doing two jobs: enforcing a format the model mostly follows, and catching a small number of fabricated spans that would otherwise be indistinguishable from real ones.

In run B, all 452 party notes were attributed by code: 273 to a player character, 139 to the whole party, 40 to 13 companions and none unattributed, with a first name reaching its full registry name through the registry.

8.3 Quote fidelity depends on the reduce model

Map notes are span-checked, so a non-verbatim quotation in a finished document comes from the reduce step. Holding the map notes fixed and changing only the reduce model gives Table 5. The local model that was fastest at the map paraphrased inside quotation marks in the reduce. The recorded conclusion was a split: a high-volume model for the map, where code checks every note, and a faithful model for the nine small reduce calls, with a span check and an `unverified` annotation on the result.

8.4 Dossiers

Thirty NPCs were drafted in chunked mode on a local model in 39 minutes. Twenty-four passed verification and six failed, each on one or two quoted spans that paraphrased or a quotation without its citation. The 24 were published and the 6 were not. A Key NPCs section then rendered from the 24 published dossiers had all 24 lines accepted by the line check, and was correct on three points where the model-written version of the same section had been wrong (a reinstated character listed as fled, a dead one listed as alive, a player character listed as an NPC).

8.5 Negative results

Model repair. A fix pass was given the lines that code had flagged and asked to delete, move or replace them, with every action checked by code. Across three versions of its gates it deleted two correct lines in the first, one in the second and none in the third, while leaving the flagged errors mostly unfixed and once writing a placeholder label into the document. The log's finding: "detection is dependable, model fixing is not." The tracker summarises the pass as helping in about a third of cases, doing nothing in a third and hurting in a third (#505). Rule R4 is this result.

A well-formed runaway. One map call on one chapter produced 139 audit bullets, 125 of the form "X happened in a prior session" with the same in-chunk citation, walking the item list in order. Every one passed the citation check, and 98 "supported" verdicts rested on that chunk alone. A third model did the same in nearly every chunk; 42% of its output was audit text and 2,664 notes were dropped. The fixes were to require a verbatim span in every audit verdict, to judge items one at a time outside the map, and to flag outlier chunks.

A strict check that flipped an answer. A model recorded an NPC's death and cited the section by its heading instead of its key. The check dropped the row, so the earlier "alive" row won the code-built status table. The fix was to accept a section's own heading as its key by exact lookup. A dropped note is not neutral when code computes an answer from the notes that remain, which is why the drop report must be read.

Valid citation, false claim. A published dossier said an antagonist had a third simulacrum. Its citation resolved, and the cited text says a second (#506). Nothing in the mechanical checks can catch this. It was found by the experiment in the next subsection.

8.6 The reader's contract

If the documents can be wrong, how should they be read? The same preparation task was run four ways: with new or old documents, and with a skill that treats the documents as authority or as an index (verify every on-stage entity against its latest summary; the summary wins; log each disagreement). Table 6 gives the result. The index reading found eight document errors against one, at the same wall time, and its log of disagreements is a ready fix-at-source queue: it traced one stale dossier to a missing registry alias (105 unlinked mentions) and found the invented simulacrum.

The old-document arm shows that document quality still matters under verification. It marked a stale claim as verified after checking a mention that did not settle the question; the new-document arm got it right because an annotation pointed at the latest evidence.

8.7 What was not measured

The rulings ledger, the review store, the promotion gate and the scheduler were accepted on fixture tests; we found no live-campaign measurements for them. The five historical promotion failures could not be replayed because the faulty drafts were not kept, and the gate's fixtures are labelled reconstructions. The incremental update path's replay of the real update that motivated it is recorded as not run. Post-merge audits of two of these features found defects despite passing acceptance tables (29 in one). Test totals in the pull requests are self-reported and each lists failures that predate the change.

9 Limitations

Citations resolve; they do not entail. This is the central limit and the documents say so in their own header. The audit and the annotations narrow the gap and do not close it. The system's guarantee is that a claim can be checked in one lookup, not that it has been.

The authority is itself model-assisted. The summaries are produced with a model and reviewed by a person. The pipeline trusts them. An error the review missed is faithfully cited.

One person, one domain, mostly one corpus. The reviewer is the author. The strongest results come from one campaign. Most arms were run once. The hand-verified fact set has 16 items. We have not measured reviewer time, which is the quantity the cycle design got wrong and the present design is meant to reduce.

Recall is traded away. Withholding ambiguous names, dropping uncited notes and telling the thread proposer to leave a note out all lose true information. The drop report and the withheld-forms report make the loss visible; they do not recover it.

The registry is a cost. Exact-match identity is only as good as the registry, and the registry is maintained by hand. A missing alias left one character's dossier two chapters stale.

Guards cover what they list. The syntax-tree guards protect named modules. One group of modules is model-free by inspection and not yet in a guard.

10 Beyond games

Nothing in the method depends on the subject. It needs four things from a domain.

1. *An authoritative corpus with addressable parts.* Records must be reviewed or otherwise trusted, and must divide into units small enough to cite: a scene, a clause, a dated note, a log entry.
2. *A maintainable identity registry.* Someone must be able to say which names refer to the same thing. Patients, parties to a case, services, parts and people in an organisation all have such registries already.
3. *A reader who can follow a citation.* The documents are an index. Their value depends on the reader, human or model, being held to the index contract.
4. *A person with authority to rule.* Where the record is wrong or silent, someone must be entitled to decide, and the decision must be written into the source.

The failure classes transfer directly. A clinical summary that merges two patients with similar names is class 4. A compliance summary that states a control is in place, citing a paragraph that says it is planned, is class 2. A runbook regenerated over an engineer's hand correction is class 9. The countermeasures transfer with them: exact identity, checked spans, annotation in place of rewriting, and publication bound to a signed digest.

The lesson of the first design transfers as well, and may matter more in a regulated setting than in a game. A review regime that charges full price at every stage will not be used. The price of review should follow the cost of an error at that stage; the review surface should be where the reviewer already is; and anything code can decide should be decided by code and reported, so that the person's attention is spent on what only a person can rule. In this project the game sets the

error cost low for most stages, which is why a heavy gate failed fast. A domain with a higher error cost would tolerate more ceremony, but the same accounting applies.

11 Related work

Draft note: the citations in this section have not been checked; see the note at the head of the references.

Attribution and citation. Work on attributed generation asks whether a model's statement is supported by a cited source [Rashkin et al., 2023, Bohnet et al., 2022], measures how often generative systems' citations support their sentences [Liu et al., 2023], and builds benchmarks for citing while generating [Gao et al., 2023b]. Menick et al. [2022] train a model to answer with verbatim quotations as evidence. Our system takes verbatim spans and resolvable citations as a hard filter enforced by code after generation, and reports the gap between resolving and supporting as its main residual error.

Verification and repair. Fact-level scoring [Min et al., 2023], self-consistency checks [Manakul et al., 2023], verification chains [Dhuliawala et al., 2023] and retrieve-then-revise methods [Gao et al., 2023a] use a model to check or repair a model's output. Our record argues for a narrower role: in this corpus, deterministic detection was dependable and model repair was not, and the system annotates instead of revising.

The companion papers. The survey [Roussos, 2026b] supplies the history before this paper and most of its findings on substrate: do not re-extract structure from prose rendered from it, atomised facts lose attribution, agreement is not available as a filter, and plans are not evidence. The decision-model study [Roussos, 2026c] supplies the argument for abstention over guessing and for reading rulings back. This paper adds the machinery that turns those findings into guarantees a build can enforce.

Agent memory. Tiered and reflective memory systems for agents [Packer et al., 2023, Park et al., 2023] keep model-written summaries as the agent's record. The survey's experience with such layers, and ours, is that a model-written record must be checked against its source before anything reads it.

Long-corpus synthesis. Retrieval-augmented generation [Lewis et al., 2020], graph-based corpus summarisation [Edge et al., 2024], hierarchical and incremental book-length summarisation [Chang et al., 2024] and outline-driven article writing [Shao et al., 2024] all compose model calls over a large corpus. The map and reduce structure is the classic one [Dean and Ghemawat, 2004]. What we add is a code check and a file between every pair of calls, code ownership of ordering and identity, and a human ratification step wherever a model proposes structure.

Human oversight. Guidelines for human-AI interaction [Amershi et al., 2019] stress efficient correction and scoping automation to the cost of error. Our first design is a worked example of ignoring that advice with otherwise correct machinery.

Games as a testbed. Tabletop role-playing has been proposed as a language challenge [Callison-Burch et al., 2022] with structured actual-play data [Zhu et al., 2023a] and assistants for the GM at the table [Zhu et al., 2023b]. Further work by the author uses the documents described here to bound what an NPC agent may know [Roussos, 2026d].

[TODO: Check each reference's venue and year against the published version before submission; add work on provenance in data management and on journaled recovery if a systems venue is targeted.]

12 Conclusion

We set out to build documents that are precise, accurate and verified. What we built is narrower and, we think, more useful: documents in which every line can be checked in one step, every structural decision was made by code or by a named person, and every published byte was signed. The parts a model does are the parts a model does well. The parts it does badly (deciding scope, deciding identity, repairing its own errors) were taken away one recorded failure at a time, and a test now fails the build if one comes back. The remaining error is a claim its citation does not support. The system cannot remove it, so it tells the reader where to look.

The first design had the right gates and was not used. The second moved the same gates to where the reviewer was, charged for review in proportion to the cost of error, and gave to code everything that did not need a person. That change, more than any check, is why the documents exist.

Data and code availability

The toolkit, including the pipeline, its specifications, the guard tests and the experiment write-ups cited here, is public [Roussos, 2026a]. The skills collection is public in the author's tools repository, under `dotfiles/claude/skills/` and `dotfiles/codex/skills/` [Roussos, 2026e]. The reviewed session summaries, the corpus the pipeline reads, are public [Roussos, 2026f]. They contain spoilers for running campaigns.

Use of AI assistance

The system was largely implemented by AI coding agents under the author's direction, and one of its designs, the session cycle of Section 5, was conceived by one (OpenAI's Codex agent running the Astra model). The evidence for this paper was gathered by an AI agent (Claude, Anthropic) from the repositories, the issue tracker and the experiment logs, and the agent drafted the text. The author made the decisions the paper records, supplied the account of why the session cycle stalled, and takes full responsibility for the content.

References

Unchecked references. The external references below were written from memory while drafting and have not been checked against the published works. Titles, author lists, venues and years may be wrong, and a reference may not say what the text cites it for. They will be verified before submission. The author's own works and the repository are listed correctly.

S. Amershi, D. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. Iqbal, P. N. Bennett, K. Inkpen, J. Teevan, R. Kikin-Gil, and E. Horvitz. Guidelines for human-AI interaction. In *Proceedings of CHI*, 2019.

B. Bohnet et al. Attributed question answering: Evaluation and modeling for attributed large language models. arXiv:2212.08037, 2022.

C. Callison-Burch, G. Singh Tomar, L. J. Martin, D. Ippolito, S. Bailis, and D. Reitter. Dungeons and Dragons as a dialog challenge for artificial intelligence. In *Proceedings of EMNLP*, 2022.

Y. Chang, K. Lo, T. Goyal, and M. Iyyer. BooookScore: A systematic exploration of book-length summarization in the era of LLMs. In *Proceedings of ICLR*, 2024.

J. Dean and S. Ghemawat. MapReduce: Simplified data processing on large clusters. In *Proceedings of OSDI*, 2004.

S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, and J. Weston. Chain-of-verification reduces hallucination in large language models. arXiv:2309.11495, 2023.

D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson. From local to global: A graph RAG approach to query-focused summarization. arXiv:2404.16130, 2024.

L. Gao et al. RARR: Researching and revising what language models say, using language models. In *Proceedings of ACL*, 2023.

T. Gao, H. Yen, J. Yu, and D. Chen. Enabling large language models to generate text with citations. In *Proceedings of EMNLP*, 2023.

P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.

N. F. Liu, T. Zhang, and P. Liang. Evaluating verifiability in generative search engines. In *Findings of EMNLP*, 2023.

P. Manakul, A. Liusie, and M. J. F. Gales. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of EMNLP*, 2023.

J. Menick et al. Teaching language models to support answers with verified quotes. arXiv:2203.11147, 2022.

S. Min et al. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of EMNLP*, 2023.

H. Rashkin et al. Measuring attribution in natural language generation models. *Computational Linguistics*, 49(4), 2023.

C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. arXiv:2310.08560, 2023.

J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of UIST*, 2023.

K. Roussos. CampaignGenerator: tooling for tabletop campaign management with language models. `https://github.com/kostadis/CampaignGenerator`, 2026.

K. Roussos. campaign-public: reviewed session summaries for several tabletop campaigns. `https://github.com/kostadis/campaign-public`, 2026.

K. Roussos. mytools: agent skills and supporting tools for campaign management. `https://github.com/kostadis/mytools`, 2026.

K. Roussos. Grounding documents for long-running tabletop campaigns: A practitioner's survey of twelve approaches, from LLM extraction to authored structure. Manuscript, October 2026. [TODO: arXiv id or URL].

K. Roussos. Decision models at the game table: Where non-generative classifiers help and fail in LLM tooling for tabletop role-playing campaigns. Manuscript, October 2026. [TODO: arXiv id or URL].

K. Roussos. Playing the NPC from the record: Knowledge horizons, human rulings, and backtests for language-model NPC agents in a long-running tabletop campaign. Manuscript, October 2026. [TODO: arXiv id or URL].

Y. Shao, Y. Jiang, T. A. Kanell, P. Xu, O. Khattab, and M. S. Lam. Assisting in writing Wikipedia-like articles from scratch with large language models. In *Proceedings of NAACL*, 2024.

A. Zhu, L. J. Martin, A. Head, and C. Callison-Burch. CALYPSO: LLMs as dungeon masters' assistants. In *Proceedings of AIIDE*, 2023.

A. Zhu, K. Aggarwal, A. Feng, L. J. Martin, and C. Callison-Burch. FIREBALL: A dataset of Dungeons and Dragons actual-play with structured game state information. In *Proceedings of ACL*, 2023.

A Check catalogue

B Sources for the figures in this paper

- Coverage, reduce fidelity, dossiers, fix pass, runaway, reader's contract: `experiments/20261007-chunked-state-docs/RESULTS.md`.
- Corpus and build figures: `specs/031-summary-native-grounding/session-log.md`.
- Linking and withheld forms: `specs/032-npc-dossiers-from-summaries/quickstart.md`.
- Drop tables: `specs/033-chunked-grounding-docs/quickstart.md` and `specs/034-chunked-party-planning/quickstart.md`.
- Thread name counts: `docs/cli/summary_native_howto.md`.
- Update path status: `specs/039-incremental-campaign-updates/validation.md`.
- The session cycle: branch `feat/cycle-integration`, directory `specs/campaign-cycle/`.
- Skills: `github.com/kostadis/mytools`, `dotfiles/claude/skills/*/SKILL.md` and the repository's commit history.
- Failure incidents: the project issue tracker, by number.

	Failure class	Recorded example	Countermeasure
1	The model makes a scope or attribution decision it had no evidence for	A planner chose, in three runs of three, a narrator whose player was absent and spoke zero cues (#385)	Code computes the candidate set before the call; no fallback to the full set
2	A valid citation backs an unsupported claim	A dossier stated a "third" simulacrum; the cited text says "a second" (#506)	Span check, annotation, per-item audit; reader verifies against the source
3	Broken model output is cached as success	A chunk returned 2 of 6 sections and was recorded "kept 45 dropped 0" (#515)	Outline check, one retry, non-zero exit, a drop report with reasons
4	Similarity or text shape is used as identity	A filename-prefix match sent 12,572 characters of one character's examples to every narrator; the detector was silent (#315)	Declared identity, exact match, ambiguous forms withheld, guard tests
5	Duplicated state drifts and the copy wins	A 16,303-character rulebook was stored as one line and delivered as a label (#276)	One authored source, generated projections, digests in run records
6	An instruction never reaches the model, silently	Voice specifications were absent from every render for twelve weeks (#247)	Refuse instead of warn; prove delivery with a rule that contradicts the model's default
7	One call over the whole corpus loses coverage	A single-call draft cited 22% of scenes and gave the party's level as 8; the record says 9	Chunked map, code check, code-built sections, budgeted reduce
8	Model repair of flagged errors is unsafe	A fix pass deleted a correct line and a living character across three attempts	Annotate and never rewrite; a review queue for the person
9	Generated output is read as canon, or hand work is overwritten	A consistency check treated a regenerated document as canon and flagged a correct recap	Trust tiers; drafts kept apart from documents; digest-bound publication
10	A global predicate stands in for a local condition	One "is a key set" flag was read in twelve places with four meanings (#342)	Delete the gate; each backend refuses at the call

Table 1: Failure classes from the project tracker and experiment logs. Issue numbers refer to the project tracker. Classes 1, 3, 4, 5, 6 and 7 are each supported by several independent incidents; classes 2 and 8 rest mainly on one corpus; class 10 is a single incident.

Guard	Asserts	Technique
`test_summary_native_no_llm`	28 check, build and publish modules import no model client and call no model function	syntax-tree walk
`test_annotate_never_rewrites`	removing annotations from the output returns the input, less removed player-character lines	invariant on a document that trips every detector
`test_state_docs_no_live_writes`	after a full run, the published documents are byte-identical	digest of tree
`test_state_docs_no_secrets`, `test_npc_draft_no_secrets`	a planted secret appears in no prompt and no output	canary
`test_no_writes_to_authored`	no tool writes the hand-authored directory	syntax-tree walk
`test_retrieve_render_isolation`	no function both retrieves and renders	syntax-tree walk
`test_no_prefix_identity`, `test_speaker_label_grammar`	no name-prefix or label-shape rule decides identity	source inspection
`test_delta_no_llm_in_records`	only listed functions in the update path may call a model	syntax-tree walk
`test_prompt_golden_pre_feature`	a feature that is off leaves the prompt byte-identical	frozen file

Table 2: Build-failing guards. A selection; the pipeline's test files number 97 for the core and 37 for the update path, about 3,000 collected tests (our count).

	`world_state`		`campaign_state`	
	single call	chunked	single call	chunked
Scenes cited (of 409)	91 (22%)	406 (99%)	194 (47%)	302 (74%)
Chapters cited (of 67)	54	67	65	66
Registry entities named	91	283	251	327
Quotations verbatim	17/17	140/143	5/5	16/17
NPC status rows	–	–	50	198
Characters	23K	249K	70K	141K

Table 3: Single-call baseline (Opus 5.5) against the chunked pipeline (`qwen3.8-flash-next`), first run. Between map and reduce, code kept 3,101 notes and dropped 118.

	Run A (four-document notes)	Run B (party and planning notes)
Notes kept	4,110	2,788
Notes dropped	257	231
uncited	105	200
malformed row	96	24
citation outside chunk	22	1
quoted span not found	21	3
other	13	3
Map wall time, two machines	12.6 min	955 s

Table 4: Drops by reason in two acceptance runs (`specs/033-.../quickstart.md`, `specs/034-.../quickstart.md`).

Reduce model	Quotations verbatim	Sum of call times	`world_state` size
Qwen3-Next-80B (local)	184/300 (61%)	3,260 s	208K
Sonnet 5.5, medium	44/47 (94%)	308 s	64K
Sonnet 5.5, high	50/54 (93%)	537 s	80K
Opus 5.5, low	39/49 (80%)	365 s	53K
Opus 5.5, medium	52/66 (79%)	639 s	86K

Table 5: Reduce step only, over identical cached map notes. The log attributes part of the larger model's lower score to the prompt, which allowed quotation marks around note text.

	Documents as authority		Documents as index	
	new docs	old docs	new docs	old docs
Document errors surfaced	1	0	8	5
Verification checks	–	–	36	31
Wall time (min)	6.0	5.4	5.7	5.6

Table 6: One preparation task, four configurations, one run each.

Check	Rule and failure codes
Corpus validation	Blocking: no numeric prefix, duplicate chapter, missing title, title and chapter disagree, missing scenes, empty scenes, bad scene identifier, scene in wrong chapter, duplicate scene identifier, unreadable file. Non-blocking: unknown section, possible duplicate, stale ruling, gap in range.
Citation resolution	A note's citation must resolve inside the chunk that was sent. Codes: uncited, invalid citation, outside chunk.
Verbatim span	A quotation must occur in the cited text, exactly or after folding typographic quotes. Spans under four characters are skipped.
Row grammar	Nested bullet, missing thread tag, missing world tag, malformed status row, missing party subject, malformed level row, level not in cited text.
Section completeness	A chunk missing a required section fails; one retry.
Runaway	A chunk with more than three times the median drops, and at least 20.
Identity	Exact name or alias in the entity registry; player characters excluded through `players.yaml`; threads by exact title or alias in the thread registry. An unknown status never overrides a known one.
Dossier verification	Invalid citation, outside evidence, quotation not found, citation mismatch, uncited, manual edit dropped, manual edit invalid. Advisories: typography normalised, placeholder speaker, unsupported status claim.
Arc candidates	Citation not in the notes, trigger not verbatim, states a value.
Audit verdict	Supported only with a citation inside the candidate chapters and a verbatim span of eight or more characters.
Thread grouping	Unknown kind, continues an unregistered thread, non-member, member claimed twice, empty group.
Freshness	Corpus, notes, audit, links and rulings each checked by digest; cache keys cover chapter text, prompt, backend, model and limits.
Publication	Draft digest equals composed header; verification passed; exact-draft sign-off present; target new or matching its recorded digest.
Promotion	Stale dependency, invalid document, incomplete draft, unresolved finding, sign-off missing or stale, stale destination, path conflict, incomplete claims.
Incremental edit	Target line digest, stale baseline, support not an exact source span, overlapping edits, heading, table and citation preservation.

Table 7: Deterministic checks in the pipeline.